

Testing the money logic

Windows personal finance application | Manual investigation and automated regression

My contribution. I performed manual walkthroughs, recorded application values and investigated behavior across accounts. AI assisted test planning, fixes, documentation and automated execution. The examples below come from the supplied project evidence.

75 Node tests passed

0 failed | rc.2 source snapshot

01 / Duplicate recurring income

Observed: recording the same €500 salary twice created two entries without a warning. Income rose from €1,700 to €2,200 and bank balance from €1,470 to €1,970.

Regression focus: prevent a second receipt in the same cycle, include pending receipts and allow a legitimate receipt in the next cycle.

02 / A suspected bug that was correct behavior

A pending €60 expense dated 5 October belonged to the 6 September-5 October cycle. I compared bank and budget balances before and after clearing, then checked the next cycle.

BEFORE CLEARING

Bank €1,030 / Budget €970

AFTER CLEARING

Bank €970 / Budget €970

The next cycle carried €970 with no new-period activity. Comparing the state transition with the intended rules avoided a false defect report.

03 / Savings, validation and recovery

Further work addressed mismatched savings/reserves, orphaned reserves after goal deletion, negative expense amounts, the Fresh Start workflow and backup loading. Regression checks target financial consistency and account isolation.

Evidence: supplied QA Evidence PDF and Technical Appendix; recurring-income-duplicates.test.cjs; finance-model.test.cjs; evidence/lemon-glass-source-run.txt. Manual observations: 9 Sep 2026; original manual build not recorded.

Tests that catch a defect

Lemon Glass regression + a separate Playwright portfolio demonstration

Lemon Glass 1.9.4-rc.2

The recorded 10 September source run passed 75 Node tests with zero failures. These include finance-model checks, action tests with simulated browser/storage objects and source-structure assertions. They are not 75 desktop or Playwright end-to-end tests.

Separate Playwright demonstration

A fictional engineering-request app exercises login, navigation, required fields, invalid loads, boundary values and reload persistence. The saved Chromium run reports **8 passed, 0 failed, 0 skipped and 0 flaky**. Implementation and runs were substantially AI-assisted.

CONTROLLED FAILURE / TC06

The mutation changes the minimum load from 1 N to 0 N. The app then accepts an invalid 0 N request; TC06 fails because its expected validation message is absent. The saved failure screenshot and trace document that outcome.

Boundary pairing: TC06 rejects 0, -1, 10001 and 1.5; TC07 accepts 1; TC08 accepts 10000. The mutation demonstrates that the test detects an intentionally broken rule.

How to inspect the work

Review tests/workflows.spec.js alongside test-results/results.json and mutation-results/results.json. The mutation folder includes test-failed-1.png and trace.zip. For Lemon Glass, follow the source-run log and regression files mapped in evidence/INDEX.md.

Evidence boundaries

These are saved package results, not new runs for this media. The browser rerun required intervention for server teardown; duration is not a performance result. Prepared manual cases are not newly executed passes. The rc.2 patch installation remained pending. No production, cross-browser or hosted CI coverage is claimed.

Sources: technical evidence archive, evidence/INDEX.md, Node run log and top-level Playwright JSON reports dated 10 Sep 2026.